

Semantic Chunking

Split at Paragraph & Section Boundaries

Using Regex & Semantic Splitters

A Deep-Dive Guide for RAG & NLP Engineers

Created: June 26, 2026 | 20:56

Table of Contents

1. Introduction to Semantic Chunking	3
2. Chunking Strategy Overview	3
3. Regex-Based Splitting	4
4. Regex Patterns Reference	5
5. Python Code — Regex Splitters	5
6. Semantic (Embedding-Based) Splitting	7
7. Similarity Breakpoint Types	8
8. Python Code — Semantic Splitters	8
9. LangChain SemanticChunker	10
10. Hybrid Approach: Regex + Semantic	11
11. Comparison of Chunking Methods	12
12. Integration with RAG Pipelines	12
13. Common Pitfalls & Best Practices	13
14. References	14

1. Introduction to Semantic Chunking

Semantic chunking is the practice of splitting documents at *meaningful boundaries* — paragraph breaks, section headings, or shifts in topic — rather than at arbitrary character or token counts. The goal is to produce chunks where each one represents a coherent, self-contained idea, making downstream retrieval far more accurate.

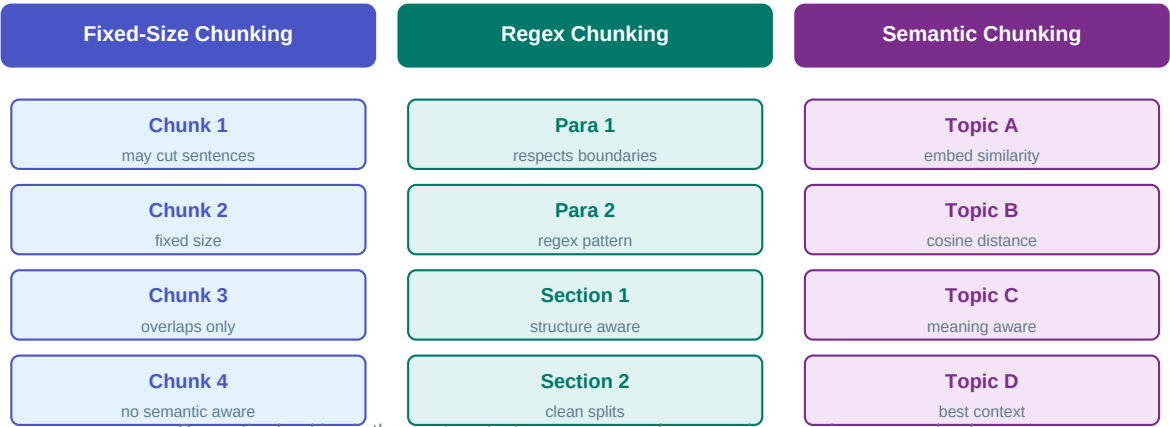
Two complementary techniques achieve this:

- **Regex-based splitting** — uses regular expressions to detect structural boundaries such as blank lines, markdown headings, or numbered sections. Fast, deterministic, and zero cost.
- **Semantic (embedding-based) splitting** — embeds each sentence, computes cosine similarity between consecutive pairs, and places boundaries wherever similarity drops below a threshold. More expensive but truly content-aware.

★ **Key Concept:** In Retrieval-Augmented Generation (RAG), the quality of retrieved chunks directly determines answer quality. Semantic chunking dramatically reduces 'noise chunks' that contain irrelevant context mixed with the answer.

2. Chunking Strategy Overview

The diagram below contrasts three major chunking strategies. Fixed-size chunking ignores content structure; regex chunking respects explicit structural markers; semantic chunking detects implicit topic shifts.



Semantic chunking is the most context-aware approach, preserving meaning across chunks.

Figure 1 — Fixed-size vs. Regex vs. Semantic chunking approaches

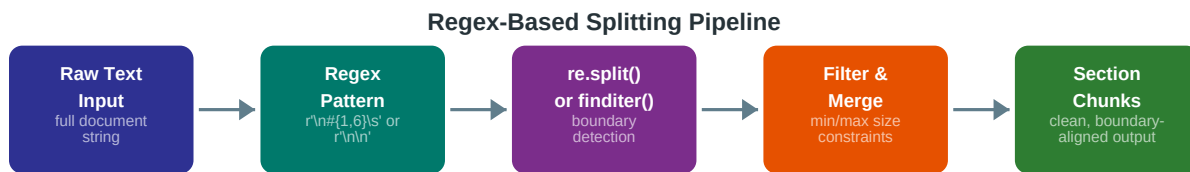
3. Regex-Based Splitting

Regular expression splitting is the simplest form of semantic chunking. Instead of cutting every N characters, you define a **pattern that matches structural boundaries** in your document format — paragraph separators, heading markers, section dividers — and split exclusively at those points.

3.1 How It Works

Python's built-in `re` module provides two key functions for boundary-aware splitting:

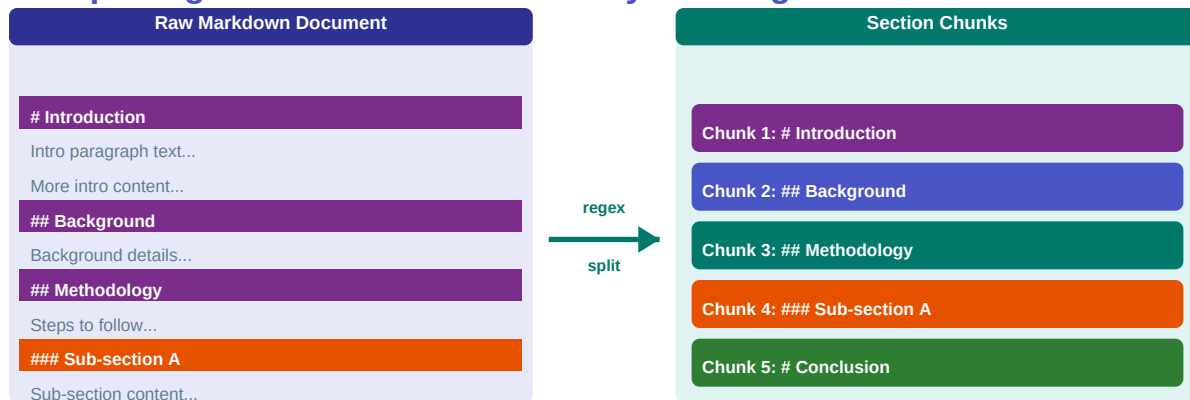
- `re.split(pattern, text)` — split at every match of the pattern.
- `re.finditer(pattern, text)` — iterate over all match positions, then slice the text manually. Useful when you need to keep boundary text.



Each step refines the raw text into clean, boundary-respecting chunks.

Figure 2 — Regex splitting pipeline from raw text to clean section chunks

3.2 Splitting a Markdown Document by Headings



Pattern: `r'\n(?:=[1,6]\s)'` splits before every heading

Figure 3 — Markdown document split into section chunks using a heading regex

The pattern `r'\n(?:=[1,6]\s)'` is a **lookahead**: it matches a newline that is immediately followed by one-to-six `#` characters and a space. Because a lookahead is zero-width, the heading itself is *not* consumed — it stays at the start of the next chunk.

4. Regex Patterns Reference

The table below lists battle-tested regex patterns for the most common document types. Combine multiple patterns by joining with | (alternation).

Pattern	Regex	Use Case
Paragraph	\n\n+	Any prose / plain text
Markdown H1	\n(?:#)	Top-level headings
Markdown any	\n(?:#{1,6}\s)	All heading levels
Numbered sec.	\n(?:\d+\.\s[A-Z])	Legal / academic docs
HR line	\n-{3,} \n={3,}	RST / Markdown dividers
Page break	\f \[PAGE\]	Scanned / Word exports
Sentence	(?<=[.!?])\s+(?=[A-Z])	Fine-grained splitting
XML/HTML tag	(?<=</(?:p div section)>)\s*	HTML documents

Table 1 — Regex patterns for common document boundaries

■ **Tip:** Combine patterns: `r'\n\n+|\n(?:#{1,6}\s)'` splits on both paragraph breaks AND heading markers, giving you the finest semantic granularity in mixed documents.

5. Python Code — Regex Splitters

5.1 Simple Paragraph Splitter

```
import re

def split_by_paragraph(text: str, min_len: int = 50) -> list[str]:
    """Split text on blank lines; drop chunks shorter than min_len."""
    chunks = re.split(r'\n\n+', text)
    return [c.strip() for c in chunks if len(c.strip()) >= min_len]

# Example usage
with open("document.txt", encoding="utf-8") as f:
    raw = f.read()

chunks = split_by_paragraph(raw)
print(f"Paragraphs found: {len(chunks)}")
for i, ch in enumerate(chunks[:3]):
    print(f"\nChunk {i+1} ({len(ch)} chars):\n{ch[:120]}...")
```

5.2 Markdown Section Splitter (preserving headings)

```
import re

def split_markdown_sections(
    text: str,
    level: int = 2,          # split on ## and above
    min_chars: int = 100,
    max_chars: int = 3000,
```

```

) -> list[dict]:
    """
    Split a Markdown document at heading boundaries.
    Returns a list of dicts with 'heading', 'content', and 'level'.
    """
    # Match any heading from H1 up to `level`
    pattern = rf'(?m)^(#{1,{level}})\s(.+)$'
    parts = re.split(pattern, text)

    sections = []
    i = 0
    while i < len(parts):
        piece = parts[i].strip()
        if not piece:
            i += 1
            continue
        # Check if it is a heading
        heading_match = re.match(r'^#{1,6}\s(.+)', piece)
        if heading_match and i + 1 < len(parts):
            heading_level = len(heading_match.group(1))
            heading_text = heading_match.group(2)
            body_text = parts[i+1].strip() if i+1 < len(parts) else ""
            combined = piece + "\n\n" + body_text
            if min_chars <= len(combined) <= max_chars:
                sections.append({
                    "heading": heading_text,
                    "level": heading_level,
                    "content": combined,
                    "chars": len(combined),
                })
            i += 2
        else:
            if len(piece) >= min_chars:
                sections.append({"heading": None, "level": 0,
                                "content": piece, "chars": len(piece)})
            i += 1
    return sections

# Usage
with open("README.md", encoding="utf-8") as f:
    md_text = f.read()

sections = split_markdown_sections(md_text, level=2)
for sec in sections:
    print(f"[H{sec['level']}] {sec['heading']} ({sec['chars']} chars)")

```

5.3 LangChain RegexTextSplitter / Custom Separators

LangChain's RecursiveCharacterTextSplitter with custom separator lists gives you regex-like structural splitting with automatic size fallback:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Markdown-aware splitter: tries headings first, falls back to paragraph, line, word
md_splitter = RecursiveCharacterTextSplitter(
    separators=[
        r"\n#{1,6}\s",      # any heading          (highest priority)
        r"\n---+",          # horizontal rule
        r"\n\n",            # paragraph break
        r"\n",              # line break
        r" ",               # word boundary
        r"",                # character (last resort)
    ],
    chunk_size=1000,
    chunk_overlap=80,
    is_separator_regex=True,
    keep_separator=True,    # keep headings at start of each chunk
)

with open("handbook.md", encoding="utf-8") as f:
    content = f.read()

chunks = md_splitter.split_text(content)
print(f"Total sections: {len(chunks)}")
print(f"Avg size: {sum(len(c) for c in chunks) // len(chunks)} chars")
```

5.4 Generic Regex Splitter with Size Guard

```
import re

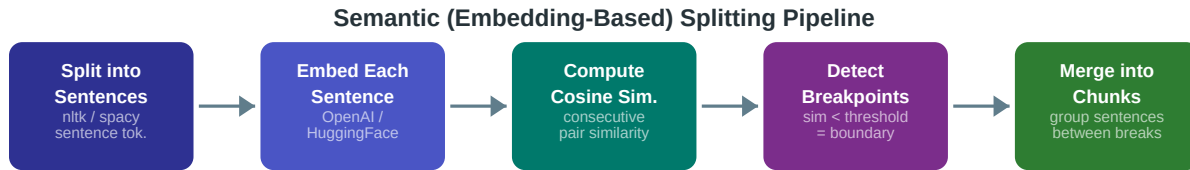
def regex_split_with_size_guard(
    text: str,
    pattern: str,
    max_chars: int = 1500,
    min_chars: int = 80,
    fallback_sep: str = "\n\n",
) -> list[str]:
    """
    Split on `pattern`. If any chunk exceeds max_chars, re-split that
    chunk on fallback_sep. Discard chunks shorter than min_chars.
    """
    raw_chunks = re.split(pattern, text)
    result = []
    for chunk in raw_chunks:
        chunk = chunk.strip()
        if not chunk:
            continue
        if len(chunk) > max_chars:
            # secondary split
            sub = re.split(fallback_sep, chunk)
            result.extend(s.strip() for s in sub if len(s.strip()) >= min_chars)
        elif len(chunk) >= min_chars:
            result.append(chunk)
    return result

# Example: split by any heading level OR paragraph
```

```
pattern = r"\n(?:#{1,6}\s)|\n\n+"
chunks  = regex_split_with_size_guard("document_text_here", pattern)
```


6. Semantic (Embedding-Based) Splitting

Semantic splitting goes beyond structure — it measures **topical similarity** between adjacent sentences using vector embeddings. A chunk boundary is placed wherever two consecutive sentences are *less similar* than a threshold, indicating a topic shift.



Chunks are formed at points where consecutive sentence embeddings are most dissimilar.

Figure 4 — Embedding-based semantic splitting pipeline

6.1 Cosine Similarity as a Boundary Signal

Given sentence embeddings $\mathbf{v}_i$ and $\mathbf{v}_{i+1}$, cosine similarity is:

$$\text{sim}(\mathbf{v}_i, \mathbf{v}_{i+1}) = (\mathbf{v}_i \cdot \mathbf{v}_{i+1}) / (||\mathbf{v}_i|| \times ||\mathbf{v}_{i+1}||)$$

Values near **1.0** mean the sentences discuss the same topic; values near **0.0** indicate a complete topic change. A boundary is inserted when the similarity drops below a chosen threshold (commonly 0.5–0.7).

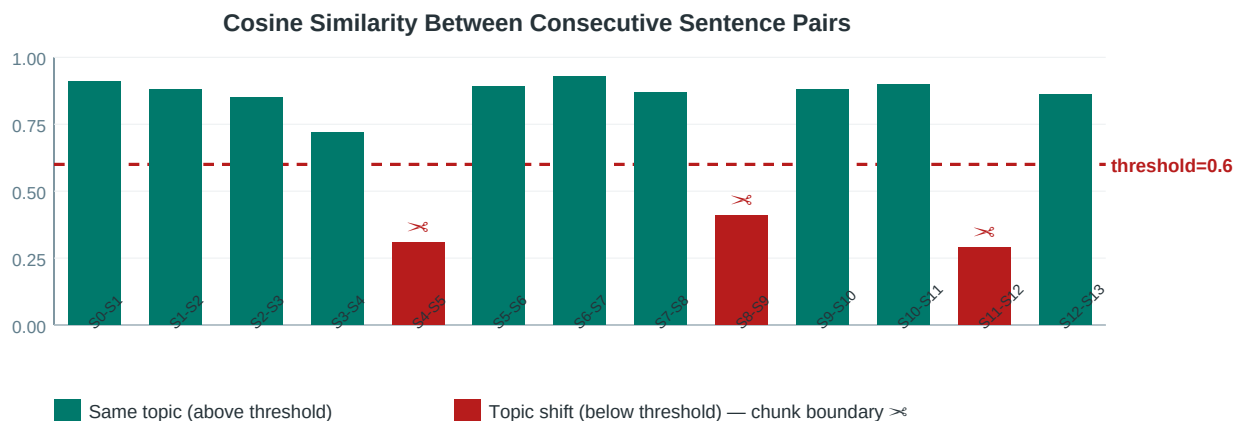


Figure 5 — Cosine similarity scores between consecutive sentence pairs. Red bars (below threshold=0.60) mark chunk boundaries (✂).

7. Similarity Breakpoint Types

Instead of a fixed similarity threshold, more sophisticated methods derive the cutoff *dynamically* from the distribution of similarity scores across the whole document. LangChain's SemanticChunker supports four strategies:

Breakpoint Type	How It Works	Threshold Param	Trade-off
Percentile	Drop at Nth-percentile similarity drop	breakpoint_threshold_amount=95	Adaptive; good default
Standard Dev.	Drop > mean - N*std below avg similarity	breakpoint_threshold_amount=3	Sensitive to outliers
Interquartile	Drop below Q1 - N*IQR of similarities	breakpoint_threshold_amount=1.5	Robust to outliers
Gradient	Second derivative of similarity curve	breakpoint_threshold_amount=95	Detects abrupt shifts

Table 2 — Breakpoint type comparison for LangChain SemanticChunker

■ **Tip:** Start with `breakpoint_threshold_type='percentile'` and `breakpoint_threshold_amount=95`. This splits at the top 5% most dramatic similarity drops — a good default for most RAG use-cases.

8. Python Code — Semantic Splitters

8.1 From-Scratch Semantic Splitter (numpy + sentence-transformers)

```
import re
import numpy as np
from sentence_transformers import SentenceTransformer

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-10))

def semantic_split(
    text: str,
    model_name: str = "all-MiniLM-L6-v2",
    threshold: float = 0.60,
    min_sentences: int = 2,
) -> list[str]:
    """
    Split `text` into semantically coherent chunks.
    A new chunk starts wherever cosine similarity between
    adjacent sentences drops below `threshold`.
    """
    # 1. Tokenise into sentences
    sentences = re.split(r'(?<=[.!?])\s+', text.strip())
    sentences = [s.strip() for s in sentences if s.strip()]

    if len(sentences) <= 1:
        return sentences

    # 2. Embed
    model = SentenceTransformer(model_name)
    embeddings = model.encode(sentences, normalize_embeddings=True)
```

```

# 3. Compute consecutive cosine similarities
sims = [cosine_similarity(embeddings[i], embeddings[i+1])
        for i in range(len(embeddings)-1)]

# 4. Detect breakpoints
breakpoints = {i+1 for i, s in enumerate(sims) if s < threshold}

# 5. Merge sentences into chunks
chunks, current = [], []
for i, sent in enumerate(sentences):
    if i in breakpoints and len(current) >= min_sentences:
        chunks.append(" ".join(current))
        current = []
    current.append(sent)
if current:
    chunks.append(" ".join(current))

return chunks

# Usage
with open("article.txt", encoding="utf-8") as f:
    article = f.read()

chunks = semantic_split(article, threshold=0.55)
print(f"Sematic chunks: {len(chunks)}")
for i, ch in enumerate(chunks):
    print(f"\nChunk {i+1} ({len(ch)} chars):\n{ch[:200]}...")

```

8.2 Percentile-Based Dynamic Threshold

```
import re
import numpy as np
from sentence_transformers import SentenceTransformer

def semantic_split_percentile(
    text: str,
    model_name: str = "all-MiniLM-L6-v2",
    percentile: float = 95,          # break at top 5% drops
) -> list[str]:
    sentences = re.split(r'(?<=[!?])\s+', text.strip())
    sentences = [s.strip() for s in sentences if s.strip()]
    model = SentenceTransformer(model_name)
    embeddings = model.encode(sentences, normalize_embeddings=True)

    sims = np.array([
        float(np.dot(embeddings[i], embeddings[i+1]))
        for i in range(len(embeddings)-1)
    ])

    # Dynamic threshold: Nth percentile of all similarity drops
    # Lower similarity = larger topic shift
    drops = 1 - sims          # convert to "distance"
    threshold = np.percentile(drops, percentile)
    breakpoints = {i+1 for i, d in enumerate(drops) if d >= threshold}

    chunks, current = [], []
    for i, sent in enumerate(sentences):
        if i in breakpoints and current:
            chunks.append(" ".join(current))
            current = []
        current.append(sent)
    if current:
        chunks.append(" ".join(current))

    print(f"Similarity stats: mean={sims.mean():.3f}, "
          f"std={sims.std():.3f}, threshold_drop={threshold:.3f}")
    return chunks
```

9. LangChain SemanticChunker

SemanticChunker is LangChain's production-ready semantic splitter. It wraps any LangChain-compatible embedding model and exposes all four breakpoint strategies through a single interface.

9.1 Installation

```
pip install langchain-experimental langchain-openai
# or for open-source embeddings:
pip install langchain-huggingface sentence-transformers
```

9.2 Basic Usage with OpenAI Embeddings

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

chunker = SemanticChunker(
    embeddings=embeddings,
    breakpoint_threshold_type="percentile",  # recommended default
    breakpoint_threshold_amount=95,         # top 5% drops = boundaries
)

with open("research_paper.txt", encoding="utf-8") as f:
    text = f.read()

chunks = chunker.split_text(text)
print(f"Chunks: {len(chunks)}")
for i, chunk in enumerate(chunks):
    print(f"Chunk {i+1} ({len(chunk)} chars): {chunk[:80]}...")
```

9.3 Using HuggingFace Embeddings (free, local)

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_huggingface import HuggingFaceEmbeddings

# Runs entirely locally – no API key needed
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

chunker = SemanticChunker(
    embeddings=embeddings,
    breakpoint_threshold_type="standard_deviation",
    breakpoint_threshold_amount=3,  # 3 std devs below mean = boundary
)

docs = chunker.create_documents([text])
# Each doc carries metadata (start_index if add_start_index=True)
for doc in docs:
    print(f"[{len(doc.page_content)} chars] {doc.page_content[:60]}...")
```

9.4 All Four Breakpoint Types

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

emb = OpenAIEmbeddings()
```

```
configs = [
    ("percentile", 95),
    ("standard_deviation", 3),
    ("interquartile", 1.5),
    ("gradient", 95),
]

for btype, bamt in configs:
    chunker = SemanticChunker(
        embeddings=emb,
        breakpoint_threshold_type=btype,
        breakpoint_threshold_amount=bamt,
    )
    chunks = chunker.split_text(text)
    avg = sum(len(c) for c in chunks) // len(chunks) if chunks else 0
    print(f"{btype:22s}: {len(chunks):3d} chunks, avg {avg} chars/chunk")
```

10. Hybrid Approach: Regex + Semantic

The most powerful production strategy combines both techniques: use regex to create coarse structural chunks (sections), then apply semantic splitting within each section to further refine at topic-shift boundaries. This keeps cost low while ensuring semantically coherent output.

Step 1 — Regex pre-split	Split the document into sections using heading or paragraph regex. Each section becomes an independent unit.
Step 2 — Size filter	Short sections (< min_chars) are kept as-is. Sections within the target range are used directly. Only long sections (> max_chars) proceed to Step 3.
Step 3 — Semantic sub-split	Apply SemanticChunker (or the from-scratch splitter) to each oversized section, breaking it further at topic-shift boundaries.
Step 4 — Merge tiny chunks	Any chunk under min_chars remaining after semantic splitting is merged with its neighbour to avoid orphan chunks that hurt retrieval.
Step 5 — Deduplicate & index	Remove near-duplicate chunks (cosine sim > 0.98) and attach metadata (source, section title, chunk index) before embedding.

10.1 Hybrid Pipeline Code

```
import re
from langchain_experimental.text_splitter import SemanticChunker
from langchain_huggingface import HuggingFaceEmbeddings

def hybrid_chunk(
    text: str,
    section_pattern: str = r"\n(?:#{1,6}\s)|\n\n+",
    min_chars: int = 100,
    max_chars: int = 2000,
    sem_threshold_type: str = "percentile",
    sem_threshold_amt: float = 95,
) -> list[dict]:
    """
    Phase 1: regex structural split.
    Phase 2: semantic sub-split for oversized sections.
    Returns list of {'text', 'chars', 'method'} dicts.
    """
    embeddings = HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-MiniLM-L6-v2"
    )
    sem_chunker = SemanticChunker(
        embeddings=embeddings,
        breakpoint_threshold_type=sem_threshold_type,
        breakpoint_threshold_amount=sem_threshold_amt,
    )

    # Phase 1
    raw_sections = [s.strip() for s in re.split(section_pattern, text)
                     if s.strip()]
    final_chunks = []
```

```

for section in raw_sections:
    if len(section) < min_chars:
        continue                # too short, skip
    elif len(section) <= max_chars:
        final_chunks.append({    # just right
            "text": section, "chars": len(section), "method": "regex"
        })
    else:
        # Phase 2: semantic sub-split
        sub = sem_chunker.split_text(section)
        for s in sub:
            if len(s.strip()) >= min_chars:
                final_chunks.append({
                    "text": s.strip(),
                    "chars": len(s.strip()),
                    "method": "regex+semantic",
                })

return final_chunks

# Run it
chunks = hybrid_chunk(open("long_document.txt").read())
for i, ch in enumerate(chunks):
    print(f"[{ch['method']:15s}] Chunk {i+1}: {ch['chars']} chars")

```


11. Comparison of Chunking Methods

Method	Mechanism	Preserves Meaning?	Cost	Best For
Fixed-Size (character)	Split every N chars	No	None	Baseline / quick prototypes
Fixed-Size (token)	Split every N tokens	Partially	Low	Token-budget-aware RAG
Regex (paragraph)	Split on <code>\n\n</code>	Mostly	None	Plain text / prose documents
Regex (section/heading)	Split on heading patterns	Yes	None	Markdown / structured docs
Semantic (embedding)	Split on cosine sim. drop	Yes	Medium	Mixed-topic, long documents
Semantic (LangChain)	SemanticChunker class	Yes	Medium	Production RAG pipelines

Table 3 — Chunking method comparison matrix

■ **Note:** For most production RAG systems, the **Hybrid (Regex + Semantic)** approach offers the best quality-to-cost ratio: regex splits are free, and semantic splits are applied only where structure alone is insufficient.

12. Integration with RAG Pipelines

Below is a complete end-to-end RAG pipeline using the hybrid chunker, FAISS vector store, and OpenAI for generation:

```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain.chains import RetrievalQA
from langchain.schema import Document

# 1. Load & chunk
raw_text = open("knowledge_base.txt", encoding="utf-8").read()
chunks = hybrid_chunk(raw_text, max_chars=1500) # from Section 10

# 2. Wrap in LangChain Documents
docs = [
    Document(
        page_content=ch["text"],
        metadata={"chunk_id": i, "method": ch["method"], "chars": ch["chars"]}
    )
    for i, ch in enumerate(chunks)
]

# 3. Embed & index
embed_model = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = FAISS.from_documents(docs, embed_model)

# 4. QA chain
```

```
llm = ChatOpenAI(model="gpt-4o", temperature=0)
qa = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vector_store.as_retriever(
        search_type="mmr",          # maximal marginal relevance
        search_kwargs={"k": 5, "fetch_k": 20},
    ),
)

# 5. Query
answer = qa.invoke({"query": "What are the main safety guidelines?"})
print(answer["result"])
```

13. Common Pitfalls & Best Practices

■ <i>Regex that swallows boundaries</i>	Using <code>re.split(r'\n\n', text)</code> removes the separator. For heading patterns, use a lookahead (<code>r'\n(?:?=#{1,6})\s'</code>) so the heading stays at the top of the next chunk.
■ <i>Threshold too low (over-splitting)</i>	Setting <code>semantic threshold=0.3</code> may produce hundreds of single-sentence chunks, destroying context. Start at 0.55–0.65 and tune upward.
■ <i>Threshold too high (under-splitting)</i>	<code>threshold=0.9</code> may never trigger a break, yielding one giant chunk that exceeds your embedding model's token limit.
■ <i>Skipping size constraints on regex splits</i>	A single regex section may span 10,000 characters. Always add a <code>max_chars</code> guard and fall back to a secondary splitter.
■ <i>Embedding cost on large corpora</i>	Semantic splitting requires one embedding per sentence. For 100K-sentence corpora this can be expensive. Use a local model (all-MiniLM-L6-v2) for chunking and a stronger model only for final indexing.
■ <i>Ignoring chunk overlap in semantic splitting</i>	<code>SemanticChunker</code> does not add overlap between chunks by default. If your retrieval system requires overlap, post-process chunks to prepend the last N sentences of the previous chunk.
■ <i>Not testing retrieval quality</i>	Always evaluate chunking by running 10–20 representative queries and checking whether retrieved chunks contain the answer. Chunk quality is the #1 factor in RAG accuracy.

13.1 Quick Best-Practice Checklist

- Use lookahead regex (`((?=...))`) to keep boundary markers in the next chunk
- Add `min_chars` and `max_chars` guards to every splitter
- Start with `SemanticChunker(breakpoint_threshold_type='percentile', breakpoint_threshold_amount=95)`
- Use a local embedding model (all-MiniLM-L6-v2) for chunking cost savings
- Apply the hybrid approach: regex first, semantic only for oversized sections
- Inspect at least 20 chunks manually before indexing a new document type
- Enable MMR (maximal marginal relevance) retrieval to reduce duplicate chunk retrieval
- Store chunk metadata (source file, section heading, chunk index) for citation
- Re-chunk when switching embedding models — chunk boundaries may shift
- Benchmark retrieval quality with a labelled question set before going to production

14. References

- [1] LangChain Docs — SemanticChunker https://python.langchain.com/docs/how_to/semantic-chunker/
- [2] LangChain Experimental — SemanticChunker source <https://github.com/langchain-ai/langchain/tree/master/libs/experimental>
- [3] LangChain — RecursiveCharacterTextSplitter (regex separators) https://python.langchain.com/docs/how_to/recursive_text_splitter/
- [4] Sentence-Transformers — all-MiniLM-L6-v2 <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- [5] OpenAI — text-embedding-3 models <https://platform.openai.com/docs/guides/embeddings>
- [6] Pinecone — Chunking Strategies for LLM Applications <https://www.pinecone.io/learn/chunking-strategies/>
- [7] Greg Kamradt — 'Chunking is All You Need' (Semantic Splitter Demo) <https://github.com/FullStackRetrieval-com/RetrievalTutorials>
- [8] Anthropic — Contextual Retrieval <https://www.anthropic.com/news/contextual-retrieval>
- [9] FAISS — Efficient Similarity Search (Meta AI) <https://github.com/facebookresearch/faiss>
- [10] Python re module — Regular Expression HOWTO <https://docs.python.org/3/howto/regex.html>

Document generated on June 26, 2026 at 20:56. Based on LangChain v0.2+, langchain-experimental 0.0.60+, Python 3.10+.